

	Departamento: Ciencias de la Computación
	Carrera: Ingeniería de Software
	Asignatura: Análisis y Diseño de Software
	NRC: 30723
	Apellidos y nombres del estudiante: Diana Guerra, Paul Moreno, Leonel Tipan

ANTECEDENTES

En el primer parcial se desarrolló el análisis y modelado de un sistema CRUD de estudiantes, cuyo propósito principal es permitir al administrador académico agregar, actualizar, eliminar y consultar registros de estudiantes mediante los datos básicos ID, nombre y edad. Este sistema fue modelado inicialmente bajo una estructura sencilla compuesta por una clase de frontera `FormularioCrudEstudiante`, una clase de control `ControlEstudiante`, una entidad `Estudiante` y un `RepositorioEstudiante` encargado de la persistencia. A partir del inicio de la unidad de patrones de diseño, se plantea mejorar este diseño aplicando patrones comportamentales, específicamente `Mediator` e `Iterator`, con el fin de reducir el acoplamiento entre componentes de interfaz, organizar mejor la comunicación interna del CRUD y controlar el recorrido de la colección de estudiantes al momento de mostrar los registros en la tabla, en concordancia con los requisitos RF-01, RF-02, RF-03, RF-04 y RF-05.

OBJETIVO

Objetivo general

Analizar, modelar e implementar los patrones de diseño comportamentales `Mediator` e `Iterator` en el sistema CRUD de estudiantes, con el propósito de mejorar la organización de la comunicación entre componentes y el recorrido de los registros almacenados.

Objetivos específicos

- Estudiar los patrones de diseño comportamentales `Mediator` e `Iterator`, identificando su definición, estructura, participantes principales, ventajas y desventajas dentro del diseño orientado a objetos.
- Analizar cómo los patrones `Mediator` e `Iterator` pueden aplicarse al sistema CRUD de estudiantes, relacionando su uso con los requisitos funcionales de agregar, actualizar, eliminar, validar y mostrar estudiantes.
- Implementar los patrones `Mediator` e `Iterator` en el CRUD de estudiantes mediante el modelado de clases y código, evidenciando cómo el mediador coordina las acciones de la interfaz y cómo el iterador permite recorrer la colección de estudiantes para mostrar los registros en la tabla.

DESARROLLO

1. Patrón Iterator

Nombre del patrón: Iterator

Definición: El patrón `Iterator` permite recorrer los elementos de una colección de manera secuencial sin exponer su estructura interna. En el proyecto, se usa para que la tabla recorra los estudiantes mediante un iterador, en lugar de depender directamente de la lista interna del repositorio.

Descripción del problema: En un CRUD de estudiantes, la tabla necesita mostrar todos los registros almacenados. Si la vista accede directamente a una lista o a la estructura interna del repositorio, se

```
@startuml
Diagrama_Iterator_CRUD_Estudiante skinparam classAttributeIconSize 0 skinparam shadowing false left to right
direction
interface "IteradorEstudiante" <<iterator>> {
+ primero()
+ haySiguiente(): boolean
+ siguiente(): Estudiante
+ actual(): Estudiante
}
interface "ColeccionEstudiantes" <<aggregate>> {
+ crearIterador(): IteradorEstudiante
+ agregar(estudiante: Estudiante)
+ eliminar(id: String)
+ obtener(posicion: int): Estudiante
+ cantidad(): int
}
```

```

class "ListaEstudiantes" <<entity>> {
-   estudiantes: List<Estudiante>
+ crearIterador(): IteradorEstudiante
+ agregar(estudiante: Estudiante)
+ eliminar(id: String)
+ obtener(posicion: int): Estudiante
+ cantidad(): int
}
class "IteradorListaEstudiantes" <<concreteIterator>> {
-   lista: ListaEstudiantes
-   posicion: int
+ primero()
+ haySiguiente(): boolean
+ siguiente(): Estudiante
+ actual(): Estudiante
}
class "FormularioCrudEstudiante" <<boundary>> {
-   txtId: String
-   txtNombre: String
-   txtEdad: String
+ clickMostrarTodo()
+ mostrarTabla(iterador: IteradorEstudiante)
+ mostrarMensaje(mensaje: String)
}
class "ControlEstudiante" <<control>> {
+ agregarEstudiante(id: String, nombre: String, edad: int): Resultado
+ actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
+ eliminarEstudiante(id: String): Resultado
+ mostrarTodos(): IteradorEstudiante
+ validarDatos(id: String, nombre: String, edad: int): boolean
} class "RepositorioEstudiante" <<repository>> {
-   coleccion: ListaEstudiantes
+ existeId(id: String): boolean
+ guardar(estudiante: Estudiante): void
+ buscarPorId(id: String): Estudiante
+ actualizar(estudiante: Estudiante): void
+ eliminar(id: String): void + listarTodos(): ColeccionEstudiantes
}
class "Estudiante" <<entity>> {
-   id: String
-   nombre: String
-   edad: int
+ Estudiante(id: String, nombre: String, edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}
class "Resultado" { - exito: boolean
-   mensaje: String
+ esExitoso(): boolean
+ getMensaje(): String
}
ListaEstudiantes ..|> ColeccionEstudiantes
IteradorListaEstudiantes ..|> IteradorEstudiante
ListaEstudiantes o-- "0..*" Estudiante : contiene
IteradorListaEstudiantes --> ListaEstudiantes : recorre
ColeccionEstudiantes --> IteradorEstudiante : crea
FormularioCrudEstudiante --> ControlEstudiante : solicita mostrar todo
ControlEstudiante --> RepositorioEstudiante : obtiene coleccion
RepositorioEstudiante o-- ListaEstudiantes : almacena
ControlEstudiante --> IteradorEstudiante : retorna iterador
FormularioCrudEstudiante --> IteradorEstudiante : recorre para mostrar tabla
ControlEstudiante --> Resultado
@enduml

```

3. Patrón Mediator

Nombre del patrón: Mediator

Definición: El patrón Mediator permite reducir las dependencias entre objetos. Este patrón restringe las comunicaciones directas entre los distintos componentes, forzándolos a colaborar e interactuar únicamente a través de un objeto central o mediador.

Descripción del problema: Se cuenta con una interfaz gráfica para gestionar la información. Esta pantalla se compone de varios controles visuales: botones para agregar, actualizar y eliminar; una tabla para mostrar todo; y campos de texto para ingresar el ID, nombre y edad del estudiante. A medida que el formulario incorpora la lógica de los requisitos funcionales, las relaciones entre estos elementos se vuelven complejas y caóticas.

- Al seleccionar un estudiante en la tabla, los campos de texto deben rellenarse automáticamente con su ID, nombre y edad.
- Al hacer clic en el botón Agregar estudiante, el sistema debe invocar primero la validación de datos revisando los campos de texto. Si es exitoso, debe actualizar la tabla para mostrar el nuevo registro y luego limpiar los campos.

Ventajas:

- Principio de responsabilidad única: permite extraer las comunicaciones complejas entre los elementos del formulario a un único lugar, haciendo el código más fácil de comprender y mantener.
- Reutilización de componentes: los componentes individuales ya no conocen ni dependen directamente de los otros controles de la interfaz, por lo que pueden utilizarse en otras pantallas vinculándolos a un nuevo mediador.
- Reducción de acoplamiento: minimiza significativamente las dependencias entre los componentes gráficos.

Desventaja:

- El anti-patrón del Objeto Todopoderoso o God Object: si el CRUD crece en funcionalidades o controles, el objeto MediatorCrudEstudiante puede acumular demasiada lógica y volverse monolítico.

¿Por qué usar el patrón Mediator? Se usa para centralizar la coordinación del formulario CRUD. Así, los botones, la tabla y los campos de texto no se comunican directamente entre sí, sino que notifican al mediador, quien decide qué operación ejecutar y cómo actualizar la interfaz.

Elemento del patrón Mediator	Clase en el proyecto	Responsabilidad
Mediator	IMediatorCrudEstudiante	Define las operaciones que centralizan la comunicación del CRUD: agregar(), actualizar(), eliminar(), mostrarTodo() y estudianteSeleccionado().
Concrete Mediator	MediatorCrudEstudiante	Coordina la comunicación entre formulario, tabla, panel de acciones y controlador. Ejecuta las operaciones CRUD según los eventos de la interfaz.
Colleague / Componente base	ComponenteCrudEstudiante	Clase base para los componentes visuales. Guarda la referencia al mediador y permite que los componentes notifiquen eventos sin depender entre ellos.
Concrete Colleague	FormularioDatosEstudiante	Maneja los campos de ID, nombre y edad. Obtiene datos, carga estudiantes seleccionados, limpia campos y muestra mensajes.
Concrete Colleague	TablaEstudiantes	Muestra la lista de estudiantes, permite obtener el ID seleccionado y limpia la tabla cuando es necesario.
Concrete Colleague	PanelAccionesEstudiante	Representa los botones de acción: agregar, actualizar, eliminar y mostrar todo. Notifica al mediador cuando ocurre un clic.
Control de apoyo	ControlEstudiante	Ejecuta la lógica de negocio del CRUD y valida los datos antes de enviar respuestas al mediador.
Repositorio de apoyo	RepositorioEstudiante	Gestiona las operaciones de almacenamiento: guardar, buscar, actualizar, eliminar y listar estudiantes.

4. Modelado Mediator

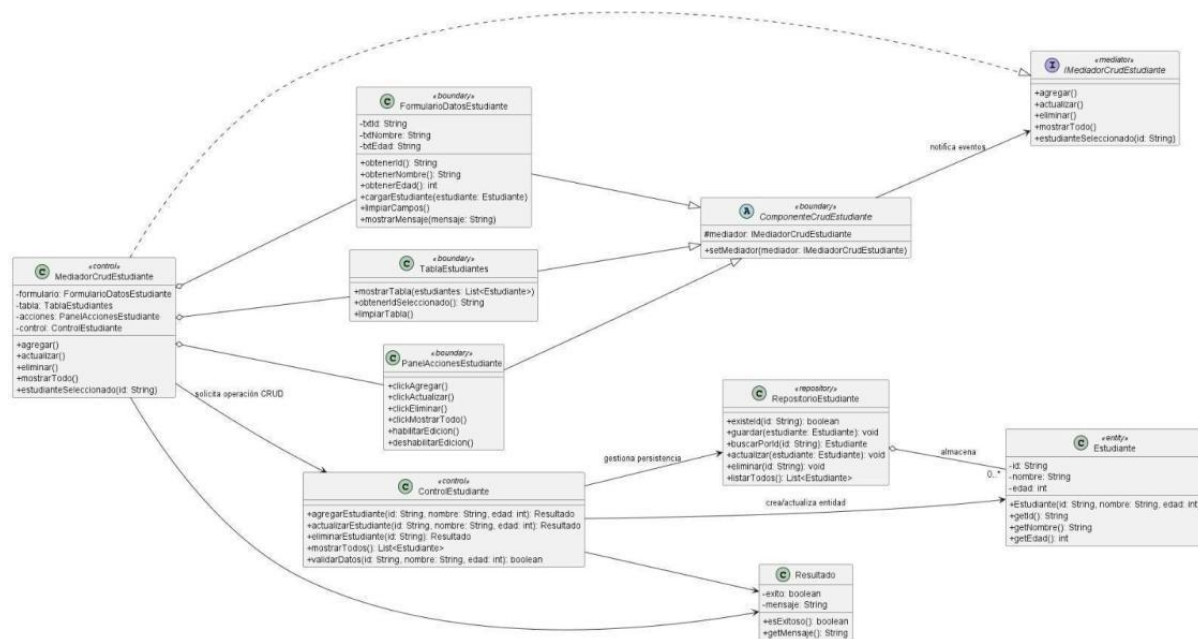


Figura 1. Diagrama UML del patrón Mediator aplicado al CRUD de estudiantes.

Código PlantUML del modelo Mediator:

```
@startuml
Diagrama_Mediator_CRUD_Estudiante
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

interface "IMediadorCruEstudiante" <<mediator>> {
    + agregar()
    + actualizar()
    + eliminar()
    + mostrarTodo()
    + estudianteSeleccionado(id: String)
}

abstract class "ComponenteCruEstudiante" <<boundary>> {
    # mediador: IMediadorCruEstudiante
    + setMediador(mediador: IMediadorCruEstudiante)
}

class "FormularioDatosEstudiante" <<boundary>> {
    - txtId: String
    - txtNombre: String
    - txtEdad: String
    + obtenerId(): String
    + obtenerNombre(): String
    + obtenerEdad(): int
    + cargarEstudiante(estudiante: Estudiante)
    + limpiarCampos()
    + mostrarMensaje(mensaje: String)
}

class "TablaEstudiantes" <<boundary>> {
    + mostrarTabla(estudiantes: List<Estudiante>)
    + obtenerIdSeleccionado(): String
    + limpiarTabla()
}

class "PanelAccionesEstudiante" <<boundary>> {
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
    + habilitarEdicion()
    + deshabilitarEdicion()
}

class "MediatorCruEstudiante" <<control>> {
    - formulario: FormularioDatosEstudiante
    - tabla: TablaEstudiantes
}
```

```

- acciones: PanelAccionesEstudiante
- control: ControlEstudiante
+ agregar()
+ actualizar()
+ eliminar()
+ mostrarTodo()
+ estudianteSeleccionado(id: String)
}

class "ControlEstudiante" <<control>> {
+ agregarEstudiante(id: String, nombre: String, edad: int): Resultado
+ actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
+ eliminarEstudiante(id: String): Resultado
+ mostrarTodos(): List<Estudiante>
+ validarDatos(id: String, nombre: String, edad: int): boolean
}

class "RepositorioEstudiante" <<repository>> {
+ existeId(id: String): boolean
+ guardar(estudiante: Estudiante): void
+ buscarPorId(id: String): Estudiante
+ actualizar(estudiante: Estudiante): void
+ eliminar(id: String): void
+ listarTodos(): List<Estudiante>
}

class "Estudiante" <<entity>> {
- id: String
- nombre: String
- edad: int
+ Estudiante(id: String, nombre: String, edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}

class "Resultado" {
- exito: boolean
- mensaje: String
+ esExitoso(): boolean
+ getMensaje(): String
}

FormularioDatosEstudiante --|> ComponenteCrudEstudiante
TablaEstudiantes --|> ComponenteCrudEstudiante
PanelAccionesEstudiante --|> ComponenteCrudEstudiante
MediadorCrudEstudiante ..|> IMediadorCrudEstudiante
ComponenteCrudEstudiante --> IMediadorCrudEstudiante : notifica eventos
MediadorCrudEstudiante o-- FormularioDatosEstudiante
MediadorCrudEstudiante o-- TablaEstudiantes
MediadorCrudEstudiante o-- PanelAccionesEstudiante
MediadorCrudEstudiante --> ControlEstudiante : solicita operación CRUD
ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena
ControlEstudiante --> Resultado
MediadorCrudEstudiante --> Resultado
@enduml

```

5. Combinación Modelo Iterator y Mediator

La combinación de los patrones permite que el Mediator coordine los eventos de la interfaz y que Iterator se encargue de recorrer la colección de estudiantes sin exponer la lista interna. De esta manera, el método `mostrarTodo()` puede solicitar los datos al controlador, recibir un iterator y enviarlo a la tabla para presentar la información.


```

- control: ControlEstudiante
+ agregar()
+ actualizar()
+ eliminar()
+ mostrarTodo()
+ estudianteSeleccionado(id: String)
}

class "ControlEstudiante" <<control>> {
+ agregarEstudiante(id: String, nombre: String, edad: int): Resultado
+ actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
+ eliminarEstudiante(id: String): Resultado
+ mostrarTodos(): IteradorEstudiante
+ validarDatos(id: String, nombre: String, edad: int): boolean
}

class "RepositorioEstudiante" <<repository>> {
- coleccion: ListaEstudiantes
+ existeId(id: String): boolean
+ guardar(estudiante: Estudiante): void
+ buscarPorId(id: String): Estudiante
+ actualizar(estudiante: Estudiante): void
+ eliminar(id: String): void
+ listarTodos(): ColeccionEstudiantes
}

interface "IteradorEstudiante" <<iterator>> {
+ primero()
+ haySiguiente(): boolean
+ siguiente(): Estudiante
+ actual(): Estudiante
}

interface "ColeccionEstudiantes" <<aggregate>> {
+ crearIterador(): IteradorEstudiante
+ agregar(estudiante: Estudiante)
+ eliminar(id: String)
+ obtener(posicion: int): Estudiante
+ cantidad(): int
}

class "ListaEstudiantes" <<concreteAggregate>> {
- estudiantes: List<Estudiante>
+ crearIterador(): IteradorEstudiante
+ agregar(estudiante: Estudiante)
+ eliminar(id: String)
+ obtener(posicion: int): Estudiante
+ cantidad(): int
}

class "IteradorListaEstudiantes" <<concreteIterator>> {
- lista: ListaEstudiantes
- posicion: int
+ primero()
+ haySiguiente(): boolean
+ siguiente(): Estudiante
+ actual(): Estudiante
}

class "Estudiante" <<entity>> {
- id: String
- nombre: String
- edad: int
+ Estudiante(id: String, nombre: String, edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}

class "Resultado" {
- exito: boolean
- mensaje: String
+ esExitoso(): boolean
+ getMensaje(): String
}

```

```

FormularioDatosEstudiante --|> ComponenteCrudEstudiante
PanelAccionesEstudiante --|> ComponenteCrudEstudiante
TablaEstudiantes --|> ComponenteCrudEstudiante
ComponenteCrudEstudiante --> IMediadorCrudEstudiante : notifica eventos
MediadorCrudEstudiante ..|> IMediadorCrudEstudiante
MediadorCrudEstudiante o-- FormularioDatosEstudiante

```



```

MediadorCrudEstudiante o-- PanelAccionesEstudiante
MediadorCrudEstudiante o-- TablaEstudiantes
PanelAccionesEstudiante --> IMediadorCrudEstudiante : clickMostrarTodo()
MediadorCrudEstudiante --> ControlEstudiante : solicita mostrar todos
ControlEstudiante --> RepositorioEstudiante : obtiene colección
RepositorioEstudiante o-- ListaEstudiantes : almacena
ListaEstudiantes ..|> ColeccionEstudiantes
IteradorListaEstudiantes ..|> IteradorEstudiante
ListaEstudiantes o-- "0..*" Estudiante : contiene
ListaEstudiantes --> IteradorListaEstudiantes : crea
IteradorListaEstudiantes --> ListaEstudiantes : recorre
ColeccionEstudiantes --> IteradorEstudiante : crearIterador()
ControlEstudiante --> IteradorEstudiante : retorna
MediadorCrudEstudiante --> TablaEstudiantes : envía iterador
TablaEstudiantes --> IteradorEstudiante : recorre para mostrar
ControlEstudiante --> Resultado
RepositorioEstudiante --> Estudiante : busca/actualiza
@enduml

```

6. Implementación en proyecto CRUD estudiante

La implementación del proyecto mantiene una separación por paquetes: controller, mediator, model y view. Esta estructura permite diferenciar la lógica de control, la coordinación del mediador, las entidades o colecciones del modelo y los componentes visuales de la interfaz.

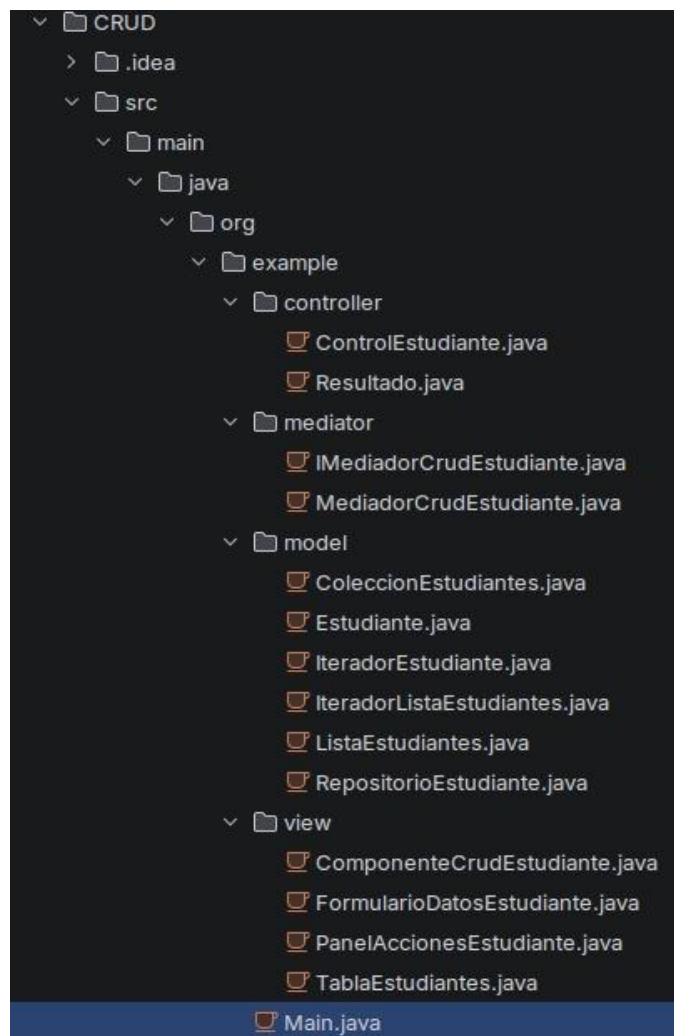


Figura 3. Estructura de paquetes y clases del proyecto CRUD.

```

src > main > java > org > example > model > IteradorListaEstudiantes.java > {} org.example.model
1  package org.example.model;
2
3  public class IteradorListaEstudiantes implements IteradorEstudiante {
4      private ListaEstudiantes lista;
5      private int posicion = 0;
6
7      public IteradorListaEstudiantes(ListaEstudiantes lista) {
8          this.lista = lista;
9      }
10
11     @Override
12     public void primero() {
13         posicion = 0;
14     }
15
16     @Override
17     public boolean haySiguiente() {
18         return posicion < lista.cantidad();
19     }
20
21     @Override
22     public Estudiante siguiente() {
23         return lista.obtener(posicion++);
24     }
25
26     @Override
27     public Estudiante actual() {
28         if (posicion == 0) return null;
29         return lista.obtener(posicion - 1);
30     }

```

Figura 4. Implementación de IteradorListaEstudiantes.

```

src > main > java > org > example > mediator > MediadorCrudEstudiante.java > {} org.example.mediator
11  public class MediadorCrudEstudiante implements IMediadorCrudEstudiante {
12      private FormularioDatosEstudiante formulario;
13      private TablaEstudiantes tabla;
14      private PanelAccionesEstudiante acciones;
15      private ControlEstudiante control;
16
17      public MediadorCrudEstudiante(ControlEstudiante control, FormularioDatosEstudiante formulario,
18          TablaEstudiantes tabla, PanelAccionesEstudiante acciones) {
19          this.control = control;
20          this.formulario = formulario;
21          this.tabla = tabla;
22          this.acciones = acciones;
23          if (this.acciones != null) this.acciones.setMediador(this);
24          if (this.tabla != null) this.tabla.setMediador(this);
25      }
26
27      @Override
28      public void agregar() {
29          Resultado r = control.agregarEstudiante(formulario.obtenerId(), formulario.obtenerNombre(), formulario.obtenerEdad());
30          formulario.mostrarMensaje(r.getMensaje());
31          mostrarTodo();
32      }
33
34      @Override
35      public void actualizar() {
36          Resultado r = control.actualizarEstudiante(formulario.obtenerId(), formulario.obtenerNombre(), formulario.obtenerEdad());
37          formulario.mostrarMensaje(r.getMensaje());
38          mostrarTodo();
39      }
40
41      @Override
42      public void eliminar() {
43          Resultado r = control.eliminarEstudiante(formulario.obtenerId());
44          formulario.mostrarMensaje(r.getMensaje());
45          mostrarTodo();
46      }
47
48      @Override
49      public void mostrarTodo() {
50          IteradorEstudiante it = control.mostrarTodos();
51          tabla.mostrarTabla(it);

```

Figura 5. Implementación de MediadorCrudEstudiante.

CRUD Estudiantes

ID:

Nombre:

Edad:

ID	Nombre	Edad
1	Diana	23
2	Leo	23
3	Paul	23

Figura 6. Interfaz final del CRUD de estudiantes.

CONCLUSIONES

1. El patrón Mediator permitió centralizar la comunicación entre los componentes de la interfaz del CRUD, evitando que botones, tabla y formulario dependan directamente entre sí.
2. El patrón Iterator permitió recorrer la colección de estudiantes sin exponer su estructura interna, facilitando que la tabla muestre los datos a partir de un iterador.
3. La combinación de ambos patrones mejora la organización del proyecto, porque separa la coordinación de eventos de la interfaz y el recorrido de la colección de estudiantes.

RECOMENDACIONES

1. Mantener el MediatorCrudEstudiante con responsabilidades claras para evitar que se convierta en un objeto demasiado grande o difícil de mantener.
2. Conservar la separación por paquetes y clases, diferenciando correctamente los componentes de vista, mediador, controlador, repositorio y modelo.
3. Documentar los métodos principales de cada clase para facilitar futuras modificaciones del CRUD y la reutilización de los patrones en otros formularios.

Sangolquí, a 26 de mayo del 2026

ANEXOS (OPCIONALES/RUBRICA DE LA GUIA)

1. Diagramas UML del patrón Mediator y de la combinación Iterator-Mediator.
2. Capturas de la implementación del proyecto CRUD de estudiantes.
3. Captura de la interfaz gráfica final del CRUD con datos de ejemplo.